

“Enterprise Parcel Management System for Courier & Parcel”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree
of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

By

SHIVAM TIWARI 2520080026

V. AKHIL 2520080029

BALAJI PATIL 2520080065

Under the Esteemed Guidance of

Dr. Divya Priya Degala

Assistant Professor

Department of Artificial Intelligence and Data Science



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE



Declaration

The Project Report entitled **“Enterprise Parcel Management System for Courier & Parcel”** is a record of bonafide work of **SHIVAM TIWARI – 2520080026, V. AKHIL – 2520080029, BALAJI PATIL – 2520080065** submitted in partial fulfillment of the requirements of the course DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E), Section 10, Team 12, during 2026–2027, Trimester 4, at KLH University. The results embodied in this report have not been copied from any other student, department, or institution.

SHIVAM TIWARI – 2520080026

V. AKHIL – 2520080029

BALAJI PATIL – 2520080065

K L (Deemed to be) University
DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE



CERTIFICATE

This is to certify that the project report entitled **“Enterprise Parcel Management System for Courier & Parcel”** is a bonafide work done and submitted by **SHIVAM TIWARI – 2520080026, V. AKHIL – 2520080029, BALAJI PATIL – 2520080065** in partial fulfillment of the requirements of the course **DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E)**, Section 10, Team 12, at KLH University during 2026–2027, Trimester 4.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success of this project would not have been possible without the timely help and guidance rendered by many people. We wish to express our sincere thanks to everyone who assisted us, in one way or the other, in the completion of our project.

Our greatest appreciation goes to our Course Coordinator **Dr. Divya Priya Degala** and our guide **Dr. Divya Priya Degala**, Department of Artificial Intelligence and Data Science, whose tremendous support, encouragement and guidance cannot be expressed in words.

We express our gratitude to **Dr. Kaja Shareef**, Head of the **Department of Computer Science and Information Technology** for providing us with adequate facilities, ways and means by which we were able to complete this project-based lab.

We thank all the members of the teaching and non-teaching staff who have assisted us directly or indirectly in the successful completion of this project. Finally, we sincerely thank our parents, friends and classmates for their kind help and cooperation during our work.

SHIVAM TIWARI – 2520080026

V. AKHIL – 2520080029

BALAJI PATIL – 2520080065

Abstract

The Enterprise Parcel Management System, built under the product name SwiftRoute, is a web-based, database-driven platform that manages courier and parcel operations from booking to final delivery. Traditional parcel handling relies on paper records, phone calls and disconnected spreadsheets, which leads to duplicated data, misplaced information, slow status updates and little visibility for customers. The proposed system replaces this with one centralized database and a role-aware web application in which customers, courier agents and administrators each see only the functions they need.

Customers can register, book a parcel, receive an automatically generated tracking number and shipping cost, pay for the shipment, and download an itemized invoice. Anyone holding a tracking number can follow the parcel's timeline without logging in. Administrators assign courier agents, monitor all shipments, onboard and suspend users, review revenue and delivery reports, read an activity audit log, and change pricing rules. Courier agents see their active manifest, record status checkpoints as a parcel moves, and close a delivery with an electronic proof of delivery containing the recipient's name, signature, photo and notes.

The application uses a React 19 and TypeScript front end with Tailwind CSS, and a Node.js and Express back end organized into routes, controllers, services and middleware. Access is protected with JSON Web Tokens, bcrypt-hashed passwords, role-based authorization and optional Google sign-in. Runtime data is held in a seven-collection datastore (users, parcels, parcel tracking, payments, delivery proofs, activity logs and system settings), while a complete third-normal-form relational design of 33 models and 30 enumerations is specified in Prisma for deployment on PostgreSQL. The system was verified with 58 automated API checks run against the real Express router and seed data; 56 passed and the remaining two exposed genuine defects, which are documented together with recommended fixes. The project demonstrates how entity-relationship design, keys and constraints, CRUD operations, role-based access and a layered backend combine into a reliable, scalable foundation for parcel logistics.

Index

S.No.	Chapters	Topics	Page No.
	Acknowledgement		4
	Abstract		5
1	Introduction	1.1 Background of the Project 1.2 Problem Statement 1.3 Scope of the Project 1.4 Objectives 1.5 Importance of the Application 1.6 Target Users / Audience	8
2	System Requirements	2.1 Hardware Requirements 2.2 Software Requirements 2.3 Development Tools and Frameworks	11
3	Technology Stack	3.1 Frontend 3.2 Backend 3.3 Database 3.4 Version Control 3.5 Authentication and Security Libraries	12
4	System Architecture	4.1 High-Level Architecture Diagram 4.2 Description of Each Layer 4.3 Deployment Architecture	13
5	Design	5.1 Database Design Principles 5.2 ER Diagram / Database Schema 5.3 Mapping from Conceptual to Implemented Design 5.4 Design Decisions and Assumptions	14
6	Implementation	6.1 Module-wise Implementation 6.2 Front-end Logic 6.3 API Integration 6.4 Authentication and Authorization	18
7	Features	7.1 List of Main Features 7.2 How Each Feature Works	21
8	Testing	8.1 API Testing Approach 8.2 Integration Testing 8.3 Tools Used for Testing 8.4 Test Cases and Results 8.5 Defects Found	25
9	Deployment	9.1 Steps to Deploy 9.2 Environment Configuration 9.3 Hosting of Frontend and Backend	28
10	Challenges & Limitations	10.1 Issues Faced During Development 10.2 Solutions Applied 10.3 Current Limitations or Known Bugs	29
11	Future Enhancements	11.1 Planned Features 11.2 Possible Integrations or Optimizations	31

S.No.	Chapters	Topics	Page No.
12	Conclusion	12.1 Summary of the Project 12.2 What Was Achieved 12.3 Skills Learned	32
13	References	Documentation, tools, and resources used	33
14	Appendices	A. Sample Code Snippets B. Installation / Setup Instructions C. Default Login Credentials D. API Reference Summary E. Screenshots F. Complete Test Results	34

CHAPTER 1: INTRODUCTION

1.1 Background of the Project

Courier and parcel services form a backbone of the modern supply chain, connecting businesses and consumers at every level. A single parcel passes through many hands: a customer who books it, a branch or warehouse that receives it, a shipment that moves it between locations, a delivery agent who carries it the last mile, and a payment record that settles the charge. In many organizations these steps are still coordinated through paper registers, phone calls and standalone spreadsheets. This works while volumes are low, but it breaks down as soon as the business grows: the same customer is typed in several times, the status of a parcel is known only to the person who last handled it, and nobody can answer “where is my parcel?” without making calls.

The Enterprise Parcel Management System, named SwiftRoute, was built to replace this fragmented way of working with a single, database-backed source of truth. Every user, parcel, tracking checkpoint, payment, delivery proof and administrative action is recorded centrally, and the system itself – rather than a person cross-checking several files – enforces who may do what, calculates shipping charges, and keeps an unbroken history of each parcel’s journey.

1.2 Problem Statement

Given a courier business that handles a continuous stream of parcels across several roles (customers, delivery agents, administrators) and several locations, the system must reliably answer four questions: who booked which parcel and what does it cost; where is a given parcel right now and what has happened to it so far; which agent is responsible for it and has it been delivered with proof; and has it been paid for. The review presentations identified the following concrete problems with manual and disconnected handling:

- Manual parcel tracking causes delays and inconsistencies in the recorded status.
- Information about customers, parcels, shipments, payments and deliveries is held in disconnected places.
- It is difficult to follow a parcel’s status across multiple locations.
- Large parcel volumes cannot be handled efficiently with manual records.
- The lack of centralized information impedes reporting and decision making.
- Different user roles need different access and visibility, which paper records cannot provide.
- Poor data organization leads to duplicate and outdated records.

Table 1.1 compares the existing manual approach with the proposed system.

Feature	Existing system	Proposed system
Booking	Manual	Automated, with tracking number and cost generated by the server

Feature	Existing system	Proposed system
Tracking	Phone-based enquiries	Real-time digital timeline, public lookup by tracking number
Records	Paper / spreadsheets	Centralized database
Errors	High	Minimal – validation and role checks in code
Speed	Slow	Fast – one click to book, assign, update or pay
Accountability	Unclear who changed what	Activity audit log for every important action

Table 1.1: Existing manual system versus the proposed system

1.3 Scope of the Project

The system covers the full life of a parcel from the point of view of the courier business: customer registration and login; booking a shipment with automatic cost calculation; assigning a courier agent; recording status changes as checkpoints; confirming delivery with electronic proof; payment and invoicing; and an administrative console for users, reports, audit logs and pricing settings. A public tracking page lets any recipient follow a parcel using only its tracking number.

The following were treated as outside the scope of this submission: real-time GPS tracking of vehicles, native mobile applications, integration with a live payment gateway (payments are recorded by the application without contacting a gateway), actual delivery of e-mail and SMS notifications, and automated route optimization. The relational design for branches, hubs and routes is specified in the production schema but is not wired into the running application. These items are listed in Chapter 11 as future enhancements.

1.4 Objectives

- **Centralized data management** – one structured database for all parcel-related information.
- **Efficient parcel tracking** – full lifecycle status visibility for customers, agents and administrators.
- **Data integrity** – accurate and consistent records, with typed models, unique tracking numbers and validated inputs.
- **Reduced data duplication** – a normalized design in which each fact is stored once.
- **Role-based access** – tailored access and dashboards for customers, agents and administrators.
- **Faster operations** – streamlined booking, assignment, status update and delivery confirmation.
- **Reporting and analytics** – useful centralized reports, revenue figures and delivery statistics.
- **Scalability** – a layered architecture and a production schema able to support growing parcel volumes.

1.5 Importance of the Application

By centralizing parcel data, the system removes the guesswork from courier operations. Customers no longer need to phone the office to learn the status of a shipment; administrators see all consignments, agents and revenue figures on one dashboard and can reassign work quickly; and agents carry a clear manifest and close each delivery with evidence. Because every action is logged and every status change leaves a checkpoint, disputes about “who did what and when” can be resolved from the record instead of from memory.

1.6 Target Users / Audience

The system serves three roles. **Customers** (shippers) book parcels, track them, pay for them and view invoices. **Courier agents** view the parcels assigned to them, update status checkpoints and submit proof of delivery. **Administrators** manage users and agents, assign parcels to agents, monitor every shipment, read reports and audit logs, and configure pricing. The public, without any account, can track a parcel with its tracking number.

CHAPTER 2: SYSTEM REQUIREMENTS

2.1 Hardware Requirements

The application is lightweight. The Node.js back end and the file-based datastore handle the data volume of a small courier operation comfortably, so the entire stack – front end, back end and datastore – can run on one computer. A development or demonstration machine with a dual-core processor, 4 GB of RAM and about 1 GB of free disk space (mainly for the installed Node modules and the project source) is sufficient. A PostgreSQL server, if the optional production schema is deployed, is an additional requirement.

2.2 Software Requirements

- Node.js 20 or later with npm (the verification described in Chapter 8 was run on Node.js 22)
- A modern web browser (Chrome, Edge or Firefox)
- Git, for version control
- Windows PowerShell, for the convenience scripts supplied with the project (swiftroute.ps1, start-clean.ps1, test-app.ps1); the application itself is platform independent
- PostgreSQL 14 or later (optional – only for the Prisma production schema)
- Visual Studio Code and Postman (recommended for development and API testing)

2.3 Development Tools and Frameworks

React 19 with TypeScript, bundled by Vite, was used for the front end, and Tailwind CSS 4 for styling. The back end is a REST API written in TypeScript on Node.js with the Express framework and executed in development with tsx. Prisma 6 is used to specify the production relational schema. Git and GitHub were used for source control, VS Code as the primary editor, and Postman for API testing during development. Table 2.1 summarizes the toolset.

Category	Technology / software	Purpose
Frontend	React 19, TypeScript, Vite, Tailwind CSS 4	User interface for all three roles
Backend	Node.js, Express 4, TypeScript (tsx)	REST API, business logic
Database	JSON datastore (runtime); Prisma + PostgreSQL (production schema)	Store and manage all parcel-related data
Security	jsonwebtoken, bcryptjs, Passport (Google OAuth 2.0)	Authentication and authorization
API testing	Postman; scripted HTTP client	Test backend APIs
IDE	Visual Studio Code	Development and coding
Version control	Git and GitHub	Source-code management
Operating system	Windows	Development platform

Table 2.1: Frameworks, platforms and software used

CHAPTER 3: TECHNOLOGY STACK

3.1 Frontend

The front end is a single-page application built with React 19 and TypeScript and bundled with Vite. Tailwind CSS 4 provides utility-based styling, including a dark mode, and the motion library (Framer Motion) provides animated transitions. Lucide supplies the icon set and canvas-confetti a small celebratory effect on successful booking. Application state that must be shared – the signed-in user (AuthContext) and the light or dark theme (ThemeContext) – is held in React Context; all other state stays local to the component that needs it. All HTTP calls are made through a single service module (src/services/api.ts) so that no page hard-codes a URL.

3.2 Backend

The back end is a REST API built with Node.js and Express and written in TypeScript. It is organized into routes (URL and required role), controllers (request and response handling and input validation), services (business rules such as pricing, assignment, status changes and payments), middleware (authentication, role checks, input sanitizing, logging and error handling) and utilities (tracking-number generation, password hashing, standard response format). This separation keeps every file focused on one responsibility.

3.3 Database

Two complementary database artefacts are part of the project. At run time the application uses a file-backed datastore (backend/database/datastore.json) holding seven collections – users, parcels, parcel_tracking, payments, delivery_proofs, activity_logs and system_settings – loaded into memory at start-up and written back to disk after every change. It is seeded automatically on first run, which makes the project easy to install and demonstrate. For production use, a complete relational schema is specified in prisma/schema.prisma: 33 models and 30 enumerations designed to third normal form, with primary and foreign keys, unique constraints, referential-action rules, soft-delete columns and indexes on the columns used for lookup, filtering and sorting. It targets PostgreSQL and can be generated and seeded with the supplied Prisma scripts.

3.4 Version Control

Git was used throughout development with the project hosted on GitHub, so that the three team members could work on the front end, back end and database design in parallel and merge their work. The repository holds the front end (src), back end (backend), shared type definitions (shared), the Prisma schema and seed (prisma), and the documentation and helper scripts.

3.5 Authentication and Security Libraries

Passwords are hashed with bcryptjs (10 salt rounds) before being stored; plain-text passwords are never written to the datastore. Sessions are stateless JSON Web Tokens (JWT) signed with a server-side secret, valid for seven days and carrying only the user's id, e-mail and role. Express middleware verifies the token on every protected route, loads the user, rejects

suspended accounts and enforces role-based access, so a customer token can never reach an admin-only endpoint. Passport with the Google OAuth 2.0 strategy provides optional “Sign in with Google”. Cross-origin access is restricted with CORS, and string inputs are trimmed by a sanitizing middleware.

CHAPTER 4: SYSTEM ARCHITECTURE

4.1 High-Level Architecture Diagram

The system follows a standard layered architecture that separates the user interface, the business logic and the stored data. Figure 4.1 shows the layers and how a request flows between them.

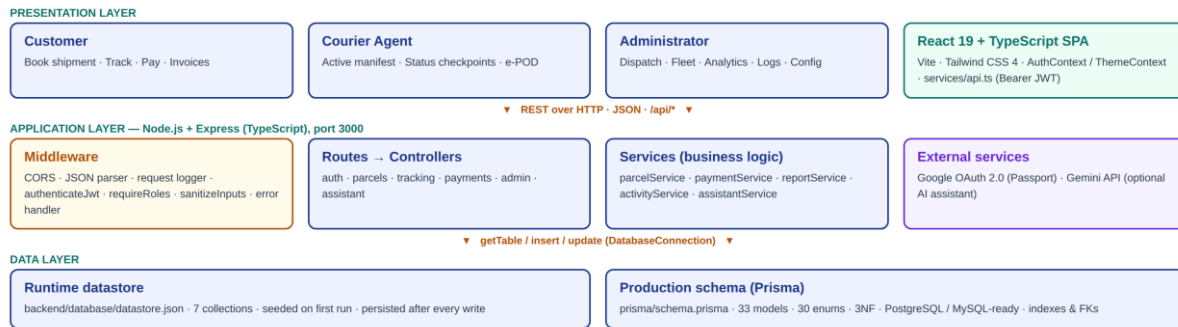


Figure 4.1: High-level layered architecture of the Enterprise Parcel Management System

4.2 Description of Each Layer

Presentation layer. The React single-page application never touches the data store directly; everything it displays comes from calling the REST API under /api. It consists of a public landing page with a tracking box and a rate calculator, a sign-in and registration page, and three role-specific dashboards (customer, courier agent, administrator) that share a common layout, navigation, status badges, tracking timeline and modal components. The signed-in user’s token is attached to every request automatically.

Application layer. Each request passes through middleware – CORS, JSON body parsing, a request logger – and then the route’s own chain: authenticateJwt verifies the token and loads the user, requireRoles checks the role, and sanitizeInputs trims input. The controller validates required fields and calls a service, which applies the business rules (for example, calculating the shipping cost from the current system settings, or refusing a second payment for an already-paid parcel) and reads or writes data. Every response uses one envelope – success, message, data and optional meta – and meaningful HTTP status codes (201, 400, 401, 403, 404, 409, 422, 500).

Data layer. A DatabaseConnection class exposes generic getTable, insert and update operations over the seven collections, keeping them in memory and persisting them to datastore.json after each write. The production schema in Prisma describes the same domain, expanded to 33 models, for deployment on PostgreSQL.

4.3 Deployment Architecture

For development and demonstration, one Node.js process serves everything on port 3000: the Express API under /api and, in development, the React application through Vite running in middleware mode (a separate HMR WebSocket port is used by Vite). For a production-style run, npm run build compiles the front end into dist and bundles the server into dist/server.cjs,

which then serves the static front end and the API from the same port. No containerization or continuous-integration pipeline was configured for this submission; Chapter 11 outlines how it could be extended to a hosted environment.

CHAPTER 5: DESIGN

5.1 Database Design Principles

The design follows the normalization principles taught in the course. Each fact is stored in one place and relationships are expressed through keys rather than by copying data. Parcels refer to their sender and agent by user id; tracking checkpoints, payments and delivery proofs each refer to their parcel by parcel id; and the pricing rates live in one settings record rather than inside every parcel formula. In the production schema this is taken to third normal form: addresses are separated from customers and parcels, vehicle specifications belong to an AgentVehicle rather than to the agent or parcel, hub capacity and coordinates are separated from routes, and shipping tariffs are held in versioned parcel categories. Constraints – primary keys, foreign keys with explicit delete behaviour, unique columns such as e-mail, tracking number, invoice number and payment reference, and indexes on frequently searched columns – are declared in the schema so that the database, not only the application code, protects integrity.

5.2 ER Diagram / Database Schema

Runtime datastore. Figure 5.1 shows the seven collections that the running application uses and the relationships between them. Table 5.1 states the purpose of each.

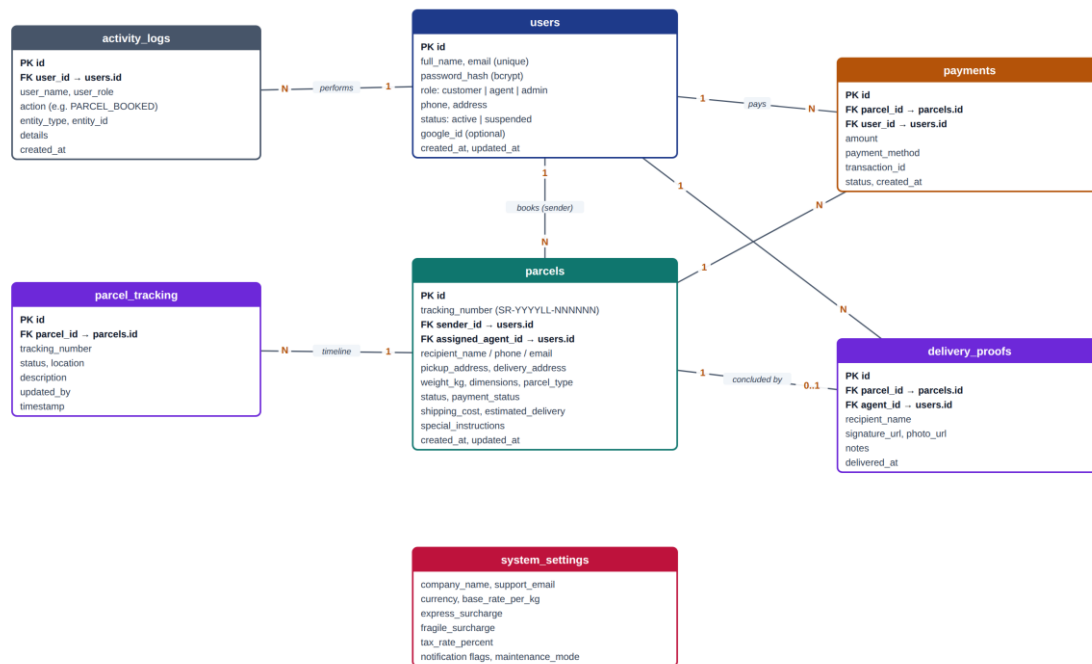


Figure 5.1: Entity-relationship diagram of the seven runtime collections

Collection	Purpose and key points
users	Every account (customer, agent or admin) with a bcrypt password hash, role and active/suspended status; optional Google id.
parcels	The core shipment record: unique tracking number, sender, assigned agent, recipient, addresses, weight, type, status, payment status, cost and estimated delivery.

Collection	Purpose and key points
parcel_tracking	Append-only timeline: one checkpoint (status, location, description, who, when) for each change in a parcel's life.
payments	One record per settled payment: parcel, payer, amount, method and transaction id.
delivery_proofs	Electronic proof of delivery: recipient name, signature, photo, notes, agent and time; at most one per parcel.
activity_logs	Audit trail of important actions (booking, assignment, status update, payment, delivery, settings changes).
system_settings	Single record of company details, currency, base rate per kg, express and fragile surcharges, tax rate and feature flags.

Table 5.1: Runtime collections and their purpose

Production schema. Figure 5.2 gives a simplified map of the Prisma schema, and Table 5.2 lists its ten modules. In the production design a user may be specialised as a Customer or a DeliveryAgent; parcels are classified by ParcelCategory, move between Branches and Hubs along Routes, are assigned to agents through ParcelAssignment, and are tracked through TrackingHistory; billing is split into Invoice, InvoiceLineItem, Payment and TransactionHistory.

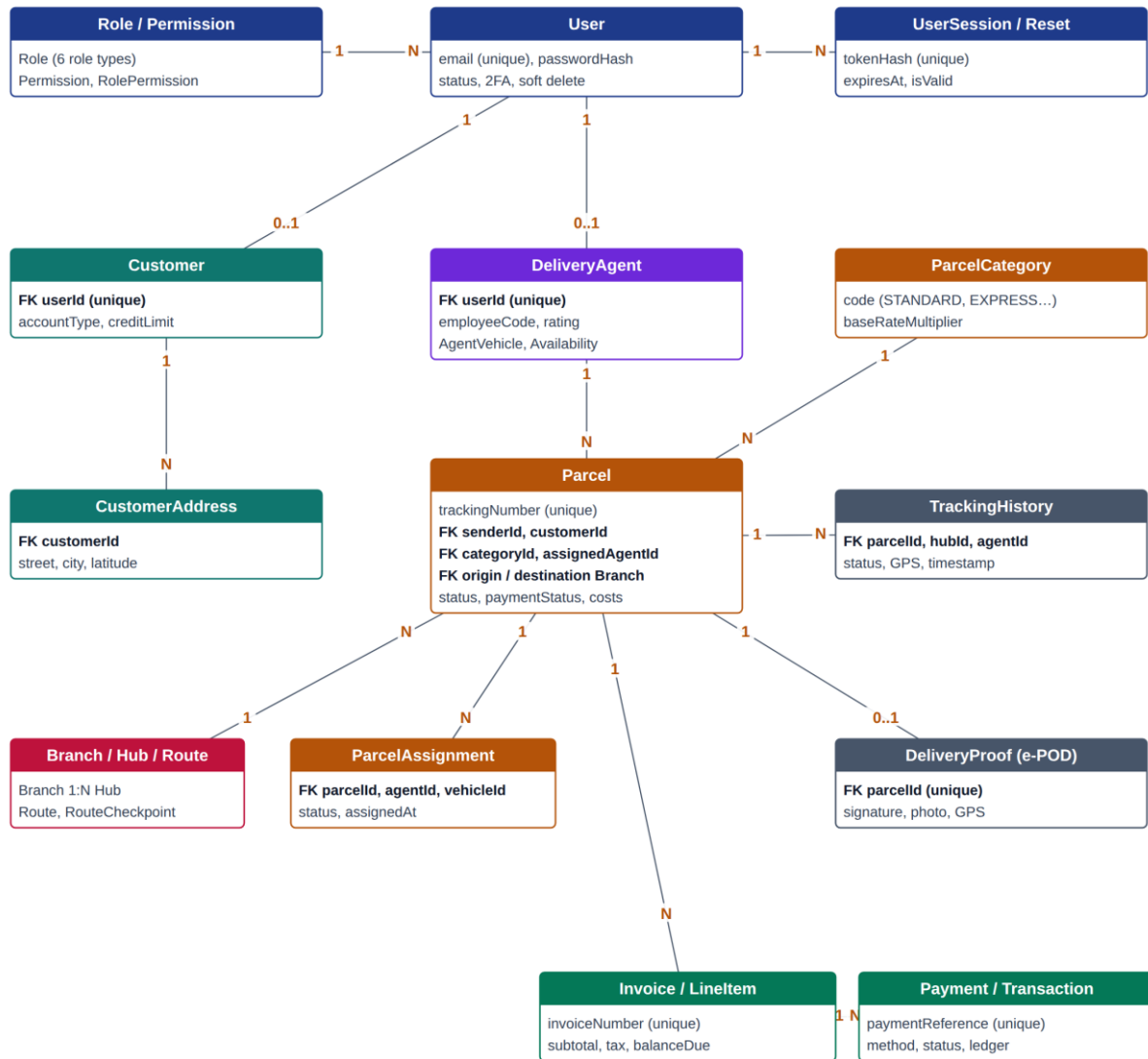


Figure 5.2: Simplified entity map of the production Prisma schema (main entities of the 33 models)

Module	Models
1. Authentication, users and RBAC	User, Role, Permission, RolePermission, UserSession, PasswordReset
2. Customer management	Customer, CustomerAddress
3. Delivery agents and fleet	DeliveryAgent, AgentVehicle, AgentAvailability
4. Logistics infrastructure	Branch, Hub, Route, RouteCheckpoint
5. Parcel lifecycle and tracking	ParcelCategory, Parcel, ParcelAssignment, TrackingHistory, DeliveryProof
6. Billing and payments	Invoice, InvoiceLineItem, Payment, TransactionHistory
7. Notifications	Notification, NotificationPreference
8. Customer support	SupportTicket, TicketMessage
9. Audit and configuration	ActivityLog, AuditLog, SystemSetting
10. Reporting roll-ups	DailyOperationalMetric, AgentPerformanceMetric

Table 5.2: Modules of the production schema (33 models, 30 enumerations)

5.3 Mapping from Conceptual to Implemented Design

The review presentations described a conceptual design of eight entities – Customer, Parcel, Shipment, Warehouse, Employee, Delivery, Payment and Tracking. During implementation these were refined. Table 5.3 shows where each concept lives in the final system, so that the early ER diagram and the delivered design can be read together.

Conceptual entity	Runtime implementation	Production schema
Customer	users (role = customer)	Customer, CustomerAddress
Parcel	parcels	Parcel, ParcelCategory
Shipment	Status, estimated_delivery and checkpoints on the parcel	Route, RouteCheckpoint, ParcelAssignment
Warehouse	Location text recorded on each checkpoint	Branch, Hub
Employee	users (role = agent)	DeliveryAgent, AgentVehicle
Delivery	Agent fields on parcels and delivery_proofs	ParcelAssignment, DeliveryProof
Payment	payments; invoice generated on demand	Invoice, InvoiceLineItem, Payment, TransactionHistory
Tracking	parcel_tracking	TrackingHistory

Table 5.3: Conceptual entities and their implementation

5.4 Design Decisions and Assumptions

- **One users collection with a role field** at run time keeps authentication and role checks simple; the production schema separates the role-specific data into Customer and DeliveryAgent profiles.
- **Append-only tracking.** The current state of a parcel is held in parcels.status, while every change also inserts a checkpoint into parcel_tracking, so the full history is never overwritten.
- **Server-side pricing.** The cost of a parcel is always computed on the server from the settings record; the client never supplies a price. $\text{Cost} = \max(5.00, \text{weight} \times \text{base rate})$ plus a surcharge by type (express 15.00, fragile 7.50, heavy 20.00); tax is added on the invoice.
- **Estimated delivery** is set to 24 hours after booking for express parcels and 72 hours for all others.
- **One payment settles the parcel.** A payment records the full shipping cost; a second payment attempt for a paid parcel is refused. Payments are recorded by the application without contacting an external gateway.
- **Public tracking.** The tracking endpoint is open, with the tracking number acting as the access key, so recipients need no account.
- **Suspension instead of deletion.** Accounts are deactivated by setting their status to suspended, which blocks sign-in and token use while preserving history.

CHAPTER 6: IMPLEMENTATION

6.1 Module-wise Implementation

The back end is organized into six resource modules, each with its own routes and controller and backed by services: Authentication, Parcels, Tracking, Payments, Administration and the Assistant.

Authentication module

Registration validates that name, e-mail, password and phone are present, that the e-mail is well formed and the password has at least six characters, and rejects an e-mail that already exists (409). The password is hashed and a signed JWT is returned. Login compares the hash, refuses suspended accounts (403) and returns a token and the user profile. Google sign-in redirects through Passport and returns to the front end with a token.

Parcel module

Booking (customer or admin) checks the required fields (422 if missing), generates a tracking number of the form SR-YYYYLL-NNNNNN, computes the cost, sets the estimated delivery, inserts the parcel with status pending, writes the first tracking checkpoint and logs the action. Assignment (admin only) verifies that the chosen user is an agent, stores the agent on the parcel, sets the status to assigned and writes a checkpoint. A status update (agent or admin) changes the status and appends a checkpoint with a location and description. Submitting proof of delivery (agent) stores the recipient name, signature, photo and notes, sets the status to delivered and writes the final checkpoint. Listing is filtered by role – a customer sees their own parcels, an agent sees the parcels assigned to them, an admin sees all – and supports status filtering and free-text search over tracking number, recipient, address and sender. Viewing a single parcel is refused with 403 unless the requester is its sender, its agent or an admin.



Figure 6.1: Parcel status lifecycle and the actor responsible for each step

Tracking module

The public endpoint GET /api/tracking/:trackingNumber looks the parcel up case-insensitively and returns the parcel, its ordered checkpoints and the delivery proof if present; an unknown number returns 404.

Payment module

Checkout creates a payment with a generated transaction id (TXN-...), marks the parcel paid and logs the receipt; a second checkout for the same parcel is rejected. The invoice endpoint assembles company details, sender, recipient, parcel details and itemized charges – base

freight, service handling surcharge, and tax at the configured rate – and returns a total, an invoice number (INV- plus the tracking number) and the payment reference.

Administration module

Administrators can read dashboard statistics and monthly reports, list users (optionally by role), onboard a courier agent, suspend or reactivate a user (only the values active and suspended are accepted), read the activity log, and read and update the pricing settings. The whole router is protected by `authenticateJwt` followed by `requireRoles(admin)`.

Assistant module

A rule-based logistics assistant recognises tracking numbers and common intents such as tracking, navigation and parcel FAQs from a message and the user's context, and returns a reply, an optional action and follow-up suggestions. When a Gemini API key is configured the service can also call the Gemini model. The back end and the API client function are implemented; a chat widget on the front end has not yet been built.

Activity logging

Every significant service action – registration, booking, assignment, status update, delivery confirmation, payment and settings change – calls `logActivity`, which stores the actor, role, action, entity and a human-readable detail. These entries feed the administrator's audit log screen.

6.2 Front-end Logic

`App.tsx` holds a small view state (home, authentication, dashboard) and renders the dashboard that matches the signed-in user's role. After a successful sign-in the user is redirected automatically to the correct dashboard. `AuthContext` stores the JWT in the browser (key `swiftroute_token`), restores the session on page load by calling `/auth/me`, and exposes `login`, `register` and `logout`. `ThemeContext` toggles dark mode. Shared building blocks – `DashboardLayout`, `StatusBadge`, `TrackingTimeline`, `ProofModal`, `InvoiceModal`, `PaymentModal`, `SkeletonLoader`, `EmptyState`, `Breadcrumbs`, `AnimatedCounter`, `NotificationPopover` and `ThemeToggle` – are each written once and reused, so every screen loads, shows empty states and confirms actions in a consistent way. The customer dashboard has four tabs (My Consignments, Book Shipment, Settlements and Invoices, Shipper Profile), the agent dashboard two (Active Manifest Run, Delivered History) and the administrator dashboard six (Consignment Dispatch, Commercial Shippers, Fleet Couriers, Revenue Analytics, System Audit Logs, Logistics Config).

6.3 API Integration

All HTTP calls go through one service object, `api`, in `src/services/api.ts`. Its helper adds the `Content-Type` header and, if a token is stored, the `Authorization: Bearer` header to every request, and converts any non-successful response into an error carrying the server's message so that screens can show it. The base URL is the relative path `/api`, so the same build works behind the development server and the production server without configuration. Each backend

resource has matching client functions (login, register, getParcels, createParcel, updateStatus, assignAgent, submitProof, trackParcel, processPayment, getInvoice, admin functions and the assistant calls), so no component repeats fetch logic.

6.4 Authentication and Authorization

A protected route first runs `authenticateJwt`, which extracts the Bearer token, verifies its signature and expiry, loads the user from the datastore, rejects a missing user (401) or a suspended one (403), and attaches the user – without the password hash – to the request. Role-restricted routes then run `requireRoles` with the permitted roles and respond 403 with a message naming the required and current role. Table 6.1 summarizes the access rules.

Capability	Public	Customer	Agent	Admin
Track parcel by number	Yes	Yes	Yes	Yes
Book parcel	–	Yes	–	Yes
View parcels	–	Own	Assigned	All
Assign courier agent	–	–	–	Yes
Update status / submit proof	–	–	Yes	Yes
Pay / view invoice	–	Yes	Yes*	Yes
Users, reports, logs, settings	–	–	–	Yes

*Table 6.1: Role-based access rules as implemented (*payment and invoice endpoints require sign-in but, as noted in Section 8.5, do not check parcel ownership)*

Two security observations from testing are reported honestly in Chapter 8: the public registration endpoint currently accepts a role value from the request body, and payment and invoice endpoints do not verify that the requester owns the parcel. Both are straightforward to fix and are listed with recommendations.

CHAPTER 7: FEATURES

7.1 List of Main Features

Customer features

- Register and sign in (e-mail and password, or Google); edit profile
- Book a parcel by entering recipient, addresses, weight, type and special instructions; receive a tracking number and cost instantly
- Track any parcel from the public page or from “My Consignments”, with a visual timeline
- Search and filter own consignments by status
- Pay for a shipment and view or print the itemized invoice
- Use the public rate calculator on the landing page to estimate a price

Courier agent features

- See the active manifest of assigned parcels with destination, recipient contact and instructions
- Update a parcel’s checkpoint (picked up, in transit, out for delivery, failed)
- Confirm delivery with electronic proof: recipient name, signature, photo and notes
- Review delivered history

Administrator features

- Consignment Dispatch: see every parcel, filter by status, assign or reassign a courier agent, open invoices
- Commercial Shippers and Fleet Couriers: list customers and agents; onboard a new agent; suspend or reactivate accounts
- Revenue Analytics: dashboard statistics, status distribution and service-level indicators
- System Audit Logs: a time-ordered record of important actions
- Logistics Config: change company details, base rate, surcharges and feature flags that drive pricing

7.2 How Each Feature Works

Booking and pricing. When a customer submits the booking form, the server computes the shipping cost from the system settings: the greater of the 5.00 minimum charge and weight multiplied by the base rate per kilogram, plus a surcharge for express, fragile or heavy parcels. With the default settings (8.50 per kg, express 15.00, fragile 7.50), a 3 kg standard parcel costs 25.50, a 2 kg express parcel 32.00 and a 1 kg fragile parcel 16.00; these figures were confirmed by the automated tests in Chapter 8. If an administrator changes the base rate, the next booking uses it immediately.

Figure 7.1: Customer “Book Shipment” form

Public tracking and the timeline. The landing page accepts a tracking number and opens a live telemetry view with the parcel’s status badge, a progress bar (Order Booked, Picked Up, In Transit, Out for Delivery, Delivered), the estimated delivery date and the delivery proof when available. Each step on the timeline is a stored checkpoint, so nothing displayed is invented by the interface.

Figure 7.2: Public tracking view of a delivered parcel

Dispatch and status workflow. An administrator opens Consignment Dispatch, chooses a parcel and assigns an agent; the parcel becomes assigned and the agent sees it in the active manifest. The agent records each leg – picked up, in transit, out for delivery – and every update appends a checkpoint, so a parcel that goes through the whole journey carries five checkpoints (booked, assigned and three updates). When the parcel reaches the recipient, the agent submits proof of delivery, which marks it delivered.

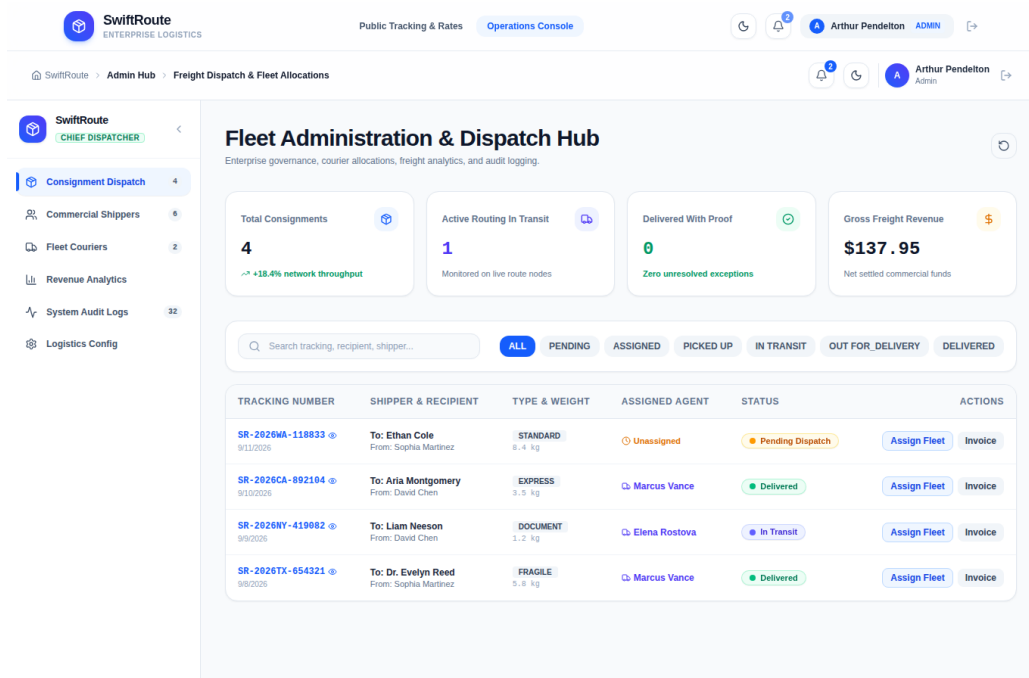


Figure 7.3: Administrator “Consignment Dispatch” with status filters and “Assign Fleet” actions

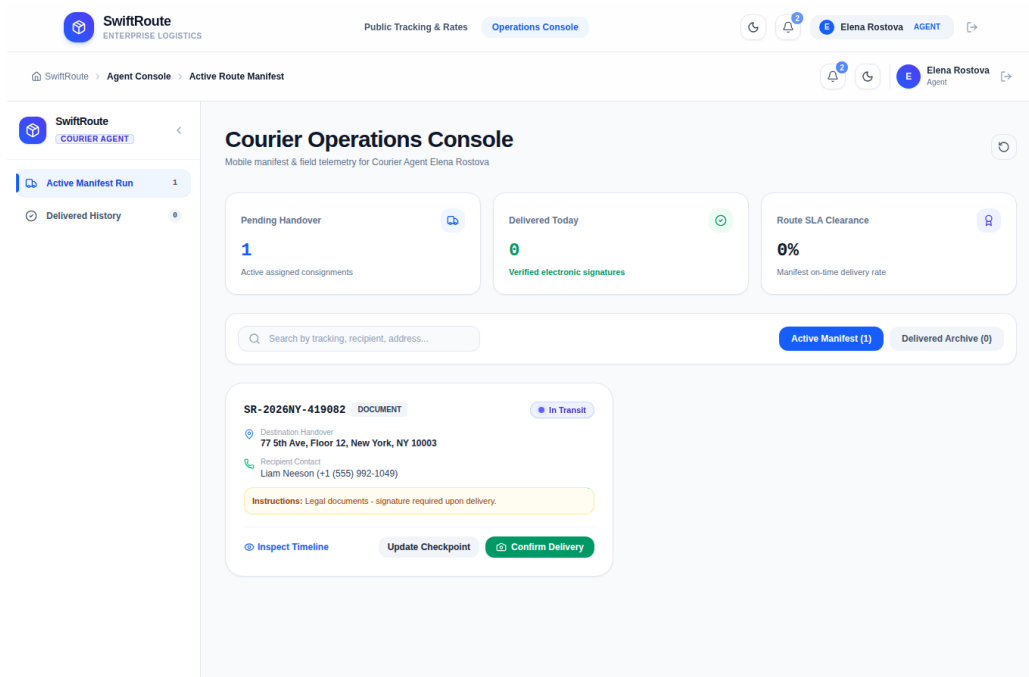


Figure 7.4: Courier agent console showing an active manifest with “Update Checkpoint” and “Confirm Delivery”

Payment and invoice. The customer chooses a payment method (card, bank transfer, cash on delivery or wallet); the server records the payment against the parcel and marks it paid. The invoice shows base freight, service handling surcharge and tax: for the 25.50 parcel above, tax at 8.25% is 2.10 and the invoice total 27.60.

Dashboard statistics. The administrator dashboard computes: total parcels; pending deliveries (status pending or assigned); in transit (picked up, in transit or out for delivery); delivered today; total revenue (sum of completed payments); active agents; total customers; and delivery success rate (delivered divided by delivered plus failed).

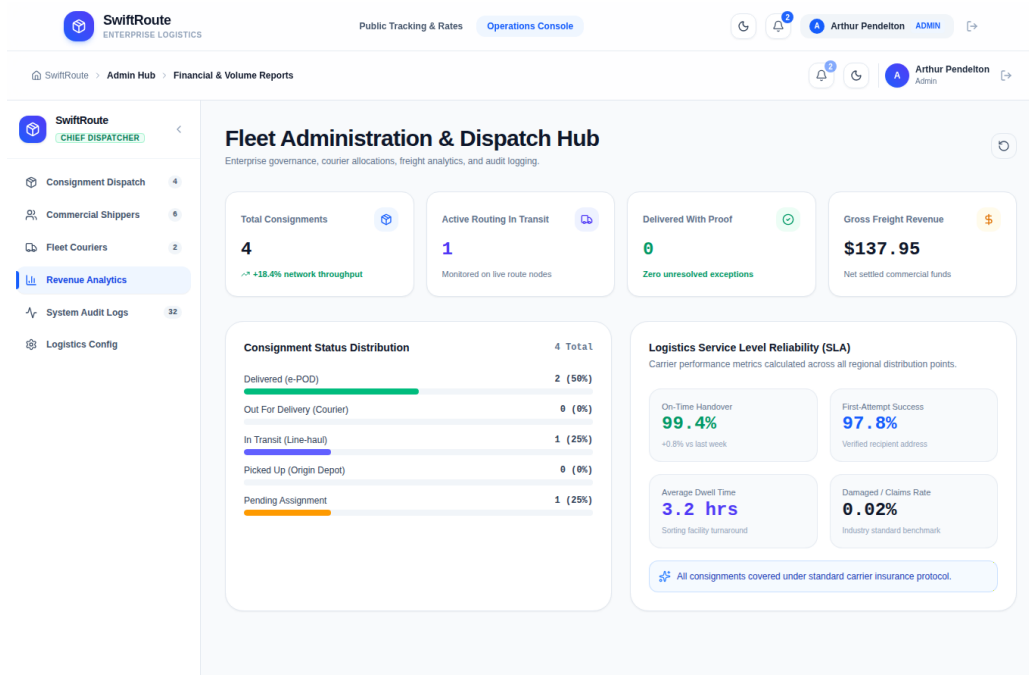


Figure 7.5: Administrator “Revenue Analytics” – status distribution and service-level indicators

Audit log and settings. Every important action creates an activity-log entry shown in the System Audit Logs tab. The Logistics Config tab edits the settings record that drives pricing and tax; changes are logged.

CHAPTER 8: TESTING

8.1 API Testing Approach

Testing concentrated on the REST API, because every business rule – pricing, authorization, status handling, payment – is enforced there. A scripted suite of 58 checks was written to exercise all six resource groups through real HTTP calls. Each check states an expected status code and, where relevant, an expected value, and records what the server actually returned. The suite logs in with the seeded customer, agent and administrator accounts, books new parcels, drives one parcel through its entire lifecycle (book, assign, three status updates, pay, invoice, proof of delivery) and then probes failure cases: wrong password, missing or invalid token, wrong role, malformed input, duplicate data, unknown ids and double payment. Postman was the planned interactive tool for the same endpoints; the scripted client was used for the recorded run because it is repeatable.

8.2 Integration Testing

The checks were executed against the real Express router with all of its middleware, controllers, services and the real datastore populated with the project's seed data – not against mocks. Because the front-end development server depends on a platform-specific Vite component that was installed for Windows, the API was mounted in a small test harness for the run; the harness imports the project's own route index unchanged. The project's built front end (the dist folder) was then served with the API and opened in a headless Chromium browser to confirm that the landing page, sign-in page, public tracking view and the customer, agent and administrator dashboards render with live data; the screenshots in this report were captured that way. In addition, the project's own status report records that its environment test script (test-app.ps1) passed 10 of 10 checks – project structure, dependencies, environment file, datastore file, TypeScript compilation, port availability, build and related checks.

8.3 Tools Used for Testing

- Scripted HTTP client (Python) – the 58-check API suite, supplied with this report as `apitest.py`
- Postman – interactive API testing during development
- TypeScript compiler (`npm run lint`) – static type checking
- Headless Chromium (Playwright) – rendering and screenshot verification of the built front end
- Manual browser testing of the three dashboards

8.4 Test Cases and Results

Of the 58 checks, **56 passed and 2 failed**. The two failures are not test errors: they uncovered real defects (Section 8.5). Table 8.1 summarizes the results by area, and Table 8.2 lists a representative selection; the complete list is in Appendix F.

Area	Checks	Passed	Failed
Authentication and registration	13	12	1
Role-based access control	7	7	0
Booking and pricing	6	6	0
Public tracking	4	4	0
Dispatch and status workflow	13	12	1
Payments and invoicing	4	4	0
Administration and settings	11	11	0
Total	58	56	2

Table 8.1: Test results by functional area

#	Test case	Expected	Actual	Result
1	Health check	200	200	Pass
2	Login with wrong password	401	401	Pass
10	Customer calls admin-only route	403	403	Pass
15	Register with duplicate email	409	409	Pass
17	Register supplying role=admin in body	403/ignored (expected secure behaviour)	201 role=admin	Fail (D1)
20	Book standard parcel (3 kg)	201, cost 25.50, status pending	201, cost 25.5, pending	Pass
22	Express parcel cost (2 kg)	17.00 + 15.00 = 32.00	32	Pass
24	Minimum charge for very light parcel (0.2 kg)	5.00	5	Pass
25	Public tracking of new parcel (no login)	200 + 1 checkpoint	200, 1 checkpoint(s)	Pass
28	New customer views another customer's parcel	403	403	Pass
33	Admin assigns courier agent	200, status assigned	200, assigned	Pass
39	Timeline has one checkpoint per transition	5 (booked, assigned, 3 updates)	5	Pass
40	Status update with value outside the allowed set	400/422 (expected secure behaviour)	200	Fail (D2)
43	Pay for the same parcel twice	400, already paid	400	Pass
44	Invoice total = cost + 8.25% tax	25.50 + 2.10 = 27.60	27.6	Pass
46	Agent submits proof of delivery	201	201	Pass
55	Suspended user login	403	403	Pass
58	New rate applied to next booking (3 kg x 10)	30.00	30	Pass

Table 8.2: Representative test cases and results

8.5 Defects Found

Testing and code review identified the following defects. They are reported here, rather than hidden, because finding them is the purpose of testing; each has a straightforward fix that is listed as future work.

ID	Defect	Evidence	Severity	Recommended fix
D1	Public registration trusts the role value in the request body, so anyone can create an administrator account.	Test 17: registering with role = admin returned 201 and an admin user.	High	Ignore the role field in /auth/register; create agents and admins only through the admin API.
D2	Status updates accept any text as the new status.	Test 40: status “banana” returned 200 and was stored.	Medium	Validate against the ParcelStatus list and allowed transitions.
D3	Assigning a non-existent agent returns 500 instead of 404.	Test 32: error returned, no data changed, wrong code.	Low	Return 404 for an unknown agent.
D4	Payment and invoice endpoints do not check that the requester owns the parcel.	Code review of paymentController and paymentService.	Medium	Compare parcel.sender_id with the caller unless the caller is an admin.
D5	Settings screen reads and writes tax_percentage, while the server stores tax_rate_percent, so the tax field appears empty.	Code review and the Logistics Config screenshot.	Low	Use the same key in the form and the API.

Table 8.3: Defects identified during testing

CHAPTER 9: DEPLOYMENT

9.1 Steps to Deploy

The application is deployed locally for development and demonstration as follows:

1. Install Node.js 20 or later and clone the repository.
2. In the project root run `npm install`.
3. Copy `.env.example` to `.env` and, if required, fill in the optional values (Section 9.2).
4. Run `npm run dev` (or the PowerShell helper `.\swiftroute.ps1 start`). The `predev` step frees ports 3000 and the Vite HMR port first.
5. Open `http://localhost:3000` in a browser and sign in with a seeded account (Appendix C).

For a production-style run, execute `npm run build`, which compiles the front end with Vite into `dist` and bundles the server with `esbuild` into `dist/server.cjs`, and then `npm start`. The datastore file is created and seeded automatically on first start; deleting it resets the system to its seed data.

9.2 Environment Configuration

Configuration is read from a `.env` file that is not committed to version control; `.env.example` documents the variables with placeholder values. They are `GEMINI_API_KEY` (optional, enables the Gemini-powered assistant), `APP_URL`, `DATABASE_URL` (PostgreSQL connection string, used only by Prisma), `GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_SECRET` and `GOOGLE_CALLBACK_URL` (optional Google sign-in), `SESSION_SECRET` and `JWT_SECRET`. The server falls back to development defaults for the secrets; these must be replaced with strong random values in any real deployment.

9.3 Hosting of Frontend and Backend

For this submission the front end and back end run as one local Node.js process on port 3000, which is appropriate for development and in-class demonstration. The README lists Vercel, Docker and a VPS with a process manager as options for hosting the same build, but a hosted deployment was not performed. Moving the runtime data from the JSON file to the Prisma and PostgreSQL schema is the main step needed before a multi-user production deployment.

CHAPTER 10: CHALLENGES & LIMITATIONS

10.1 Issues Faced During Development

- **Changing technology choices.** The abstract form listed Java with JDBC, MySQL and optionally Spring Boot, the first review proposed React, Node.js and MySQL or PostgreSQL, and the final design settled on React, Node.js, Express and TypeScript with a Prisma-specified relational schema. Keeping the documentation, diagrams and code consistent through these changes was a challenge.
- **Two data representations.** The comprehensive production schema (33 models) is much larger than the seven collections the running application needs, and the two had to be kept conceptually aligned.
- **Port conflicts.** Starting the server repeatedly produced “port already in use” errors on port 3000 and on the Vite HMR WebSocket port.
- **Post-login redirect.** After signing in, users were left on the landing page instead of their dashboard.
- **Build and type errors.** The Prisma client import failed until the client was generated, a PORT type error appeared in TypeScript, the Prisma configuration was deprecated, and seed-script enum imports needed correcting.
- **OAuth setup.** Configuring Google OAuth credentials and callback URLs required several attempts and dedicated setup notes.

10.2 Solutions Applied

- The final stack was frozen after the reviews and this report documents the conceptual-to-implemented mapping (Table 5.3).
- The production schema was kept as the target design, while a lightweight datastore with the same entities in simplified form lets the project run without installing a database server.
- A kill-ports script runs automatically before npm run dev, and start-clean.ps1 stops stray Node processes; the server port is configurable.
- The main layout now redirects an authenticated user to the dashboard for their role automatically.
- The Prisma client was generated, the port was parsed as a number, a modern prisma.config.ts replaced the deprecated settings, and the seed script was corrected.
- Step-by-step OAuth setup and verification notes (including verify-oauth-config.js) were written.

10.3 Current Limitations or Known Bugs

- The defects D1–D5 in Section 8.5, including the registration role escalation (D1), which should be fixed before any real use.

- The JSON datastore rewrites the whole file on each change and has no transactions or locking, so it suits a single-user demonstration, not heavy concurrent use.
- The production Prisma schema is specified but not connected to the running application.
- Payments are recorded without a payment gateway; there is no refund flow in the interface.
- E-mail and SMS notifications are not sent; the notification bell currently shows sample entries.
- The AI assistant exists on the server but has no chat interface yet.
- Tracking numbers are random and are not checked for uniqueness before saving.
- The six-month delivery report mixes real counts with generated baseline numbers and should be replaced by real aggregates.
- No real GPS or route optimization; location on a checkpoint is typed text.
- JWT and session secrets have development defaults.

CHAPTER 11: FUTURE ENHANCEMENTS

11.1 Planned Features

- Fix the defects in Section 8.5, beginning with the registration role escalation, status validation and parcel-ownership checks on payments and invoices.
- Real-time GPS tracking of delivery vehicles with live map display.
- E-mail, SMS and push notifications for booking, dispatch, out-for-delivery and delivery events, connected to the notification settings.
- A chat and voice interface for the existing assistant API.
- Native mobile applications for customers and couriers.
- Support tickets for damaged, delayed or lost parcels, already modelled in the production schema.

11.2 Possible Integrations or Optimizations

- Connect the runtime to the Prisma and PostgreSQL schema, using database transactions for booking, assignment and payment.
- Integrate a payment gateway such as Stripe and store its transaction records.
- Route optimization and AI-based delivery-time prediction using branch, hub and route data; demand forecasting from historical volumes.
- Real analytics: aggregate delivered, failed and revenue figures by date directly from the database.
- Containerize with Docker and add a CI pipeline for repeatable cloud deployment, with Redis or similar caching for tracking lookups at scale.
- Add an automated test suite (Jest and Supertest for the API, Playwright for the interface) built on the scripted checks used in this report.

CHAPTER 12: CONCLUSION

12.1 Summary of the Project

This project set out to replace manual, fragmented parcel handling with a single database-driven web application. Over the course of development a role-aware React front end, a 27-endpoint Express REST API, a seven-collection runtime datastore and a 33-model production relational schema were designed and built, and the working system was verified through a scripted API suite and by rendering the real interface for every role.

12.2 What Was Achieved

The finished system provides: secure authentication with hashed passwords, JWT sessions and role-based authorization; parcel booking with server-calculated pricing and generated tracking numbers; a complete, append-only tracking timeline that anyone can view by tracking number; administrator dispatch of courier agents; agent status updates and electronic proof of delivery; payment recording and itemized invoices; an administrator console with statistics, audit logs, user management and configurable rates; and a normalized production schema ready for PostgreSQL. Of 58 automated checks, 56 passed, and the two that failed were turned into documented defects with recommended fixes instead of being left unnoticed.

12.3 Skills Learned

Working on this project gave practical experience in entity-relationship modelling, keys, constraints and normalization to third normal form; translating a relational design into a Prisma schema; building a layered REST API in TypeScript with consistent response conventions and meaningful HTTP status codes; implementing JWT authentication, bcrypt password hashing and role-based access control; building a role-based React interface with Context-based state; collaborating through Git; and, importantly, systematic testing – deliberately trying to break the system’s own rules, which is how the registration and status-validation defects were found.

CHAPTER 13: REFERENCES

- React – Official Documentation, <https://react.dev>
- Vite – Official Documentation, <https://vitejs.dev>
- TypeScript Handbook, <https://www.typescriptlang.org/docs/>
- Tailwind CSS Documentation, <https://tailwindcss.com/docs>
- Node.js – Official Documentation, <https://nodejs.org/en/docs>
- Express.js – Official Documentation, <https://expressjs.com>
- Prisma ORM Documentation, <https://www.prisma.io/docs>
- PostgreSQL Documentation, <https://www.postgresql.org/docs/>
- jsonwebtoken – npm documentation, <https://www.npmjs.com/package/jsonwebtoken>
- bcryptjs – npm documentation, <https://www.npmjs.com/package/bcryptjs>
- Passport.js and passport-google-oauth20, <https://www.passportjs.org>
- Google Gemini API Documentation, <https://ai.google.dev/gemini-api/docs>
- Postman Learning Center, <https://learning.postman.com>
- MDN Web Docs – JavaScript and Web APIs reference, <https://developer.mozilla.org>

CHAPTER 14: APPENDICES

Appendix A: Sample Code Snippets

Server-side shipping cost calculation (backend/utils/trackingGenerator.ts):

```
export const calculateShippingCost = (
  weightKg: number, parcelType: string,
  baseRate = 8.5, expressSurcharge = 15.0, fragileSurcharge = 7.5
): number => {
  let cost = Math.max(5.0, weightKg * baseRate);
  if (parcelType === 'express') cost += expressSurcharge;
  if (parcelType === 'fragile') cost += fragileSurcharge;
  if (parcelType === 'heavy') cost += 20.0;
  return Math.round(cost * 100) / 100;
};
```

Role check middleware (backend/middleware/roleMiddleware.ts):

```
export const requireRoles = (...allowedRoles: UserRole[]) =>
  (req: AuthenticatedRequest, res: Response, next: NextFunction) => {
    if (!req.user) return sendError(res, 'Unauthorized: please sign in first',
401);
    if (!allowedRoles.includes(req.user.role)) {
      return sendError(res, `Access denied. Required role: [${allowedRoles.join(",
" )}]. Current role: ${req.user.role}`, 403);
    }
    next();
  };
};
```

Status update: change the status, append a checkpoint, log the action (backend/services/parcelService.ts, abridged):

```
const updated = db.update('parcels', parcelId, { status: newStatus });
db.insert('parcel_tracking', {
  parcel_id: updated.id, tracking_number: updated.tracking_number,
  status: newStatus, location, description,
  timestamp: new Date().toISOString(), updated_by: actor.full_name,
});
logActivity(actor, 'STATUS_UPDATED', 'parcel', updated.id, `...`);
```

Appendix B: Installation and Setup Instructions

1. npm install
2. copy .env.example .env (optional values only)
3. npm run dev (API and front end on http://localhost:3000)
4. npm run build, then npm start (production-style run)
5. Optional production schema: set DATABASE_URL, then npm run db:generate and npm run db:seed

Appendix C: Default Login Credentials

Demonstration accounts created automatically in the seed data (change these before any real deployment):

Role	Email	Password
Admin	admin@swiftroute.com	admin123
Courier agent	agent.marcus@swiftroute.com	agent123
Courier agent	agent.elena@swiftroute.com	agent123
Customer	customer@swiftroute.com	customer123

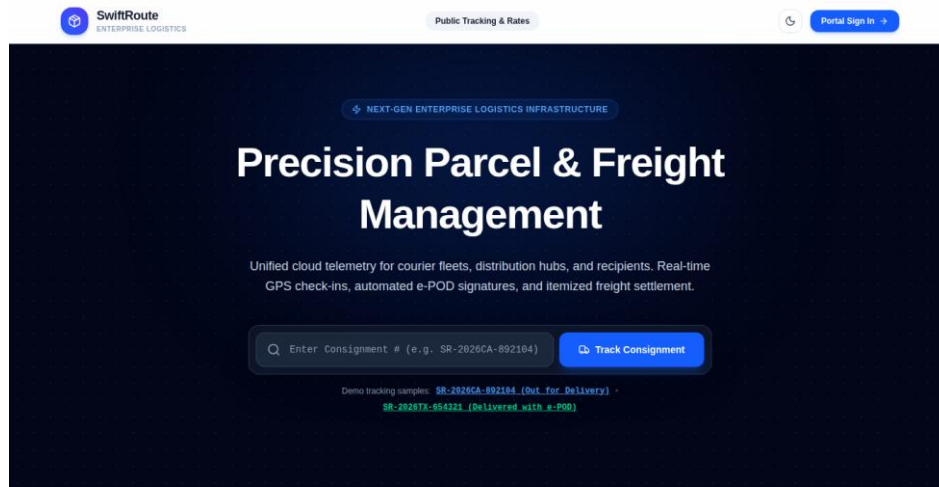
Appendix D: API Reference Summary

27 endpoints across seven groups. All bodies and responses are JSON; every response uses the envelope { success, message, data, meta }.

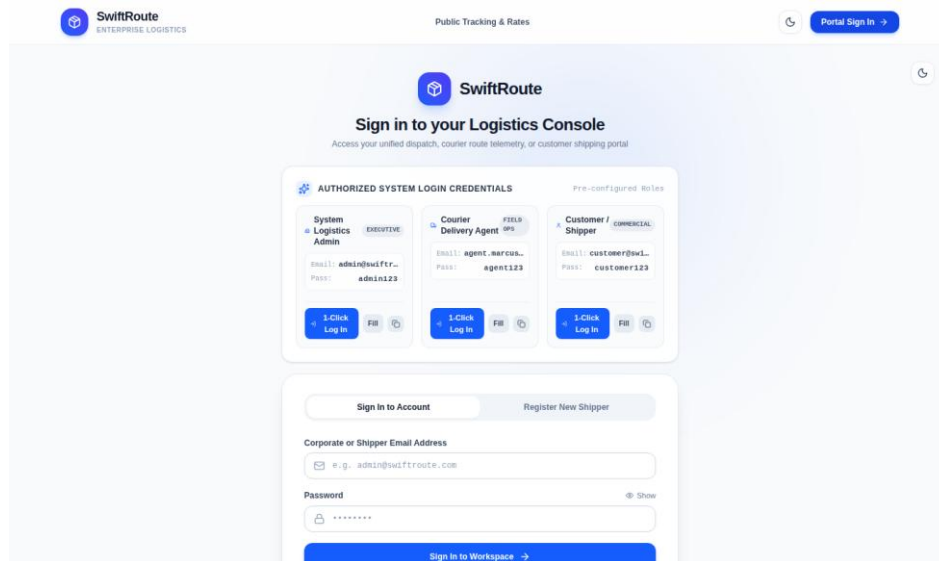
Method	Endpoint	Access	Purpose
POST	/api/auth/register	Public	Create a customer account
POST	/api/auth/login	Public	Sign in, returns JWT
GET	/api/auth/me	Signed in	Current user profile
PUT	/api/auth/profile	Signed in	Update own profile
GET	/api/auth/google	Public	Start Google sign-in
GET	/api/auth/google/callback	Public	Google sign-in callback
POST	/api/parcels	Customer, Admin	Book a parcel
GET	/api/parcels	Signed in	List parcels (filtered by role; status and search)
GET	/api/parcels/:id	Owner, Agent, Admin	Parcel detail with timeline and proof
PATCH	/api/parcels/:id/status	Agent, Admin	Update status and add checkpoint
PATCH	/api/parcels/:id/assign	Admin	Assign courier agent
POST	/api/parcels/:id/proof	Agent, Admin	Submit proof of delivery
GET	/api/tracking/:trackingNumber	Public	Track a parcel
POST	/api/payments/checkout	Signed in	Pay for a parcel
GET	/api/payments/history	Signed in	Payment records (own, or all for admin)
GET	/api/payments/invoice/:parcelId	Signed in	Invoice data
GET	/api/admin/stats	Admin	Dashboard statistics
GET	/api/admin/reports	Admin	Monthly delivery reports
GET	/api/admin/users	Admin	List users (optional role filter)
POST	/api/admin/agents	Admin	Onboard a courier agent
PATCH	/api/admin/users/:id/status	Admin	Suspend or reactivate a user
GET	/api/admin/activity-logs	Admin	Audit log
GET	/api/admin/settings	Admin	Read pricing settings
PUT	/api/admin/settings	Admin	Update pricing settings
POST	/api/assistant/chat	Optional	Assistant conversation
GET	/api/assistant/suggestions	Optional	Assistant suggestions
GET	/api/health	Public	Health check

Appendix E: Screenshots

The following screens were captured from the running application (built front end served with the API, seed data).



E.1 Landing page with tracking box (signed-out view)



E.2 Sign-in page

SwiftRoute

ENTERPRISE LOGISTICS

Public Tracking & Rates

Operations Console

David Chen

CUSTOMER

SwiftRoute

Customer Console

Active Consignments

David Chen

Customer

Book New Shipment

My Consignments

Book Shipment

Settlements & Invoices

Shipper Profile

Consignment Hub

Track live GPS progress, download commercial invoices, and manage deliveries.

Total Bookings

2

+12.5% this quarter

Active In Transit

1

Real-time GPS routing

Delivered With e-POD

1

100% handover verified

Total Freight Spend

\$69.75

Itemized commercial invoices

Search tracking, recipient, city...

ALL

IN TRANSIT

OUT FOR DELIVERY

DELIVERED

PENDING

TRACKING REFERENCE	CONSIGNEE & ROUTE	CATEGORY	FREIGHT COST T1	STATUS	ACTIONS
SR-2026CA-892104 9/9/2026	Aria Montgomery 224 Rosewood Lane, Palo Alto, CA 94301	EXPRESS 3.5 kg	\$44.75	Delivered	Invoice
SR-2026NY-419082 9/9/2026	Liam Neeson 77 8th Ave, Floor 12, New York, NY 10003	DOCUMENT 1.2 kg	\$25.00	In Transit	Invoice

E.3 Customer dashboard – My Consignments

SwiftRoute

ENTERPRISE LOGISTICS

Public Tracking & Rates

Operations Console

David Chen

CUSTOMER

SwiftRoute

Customer Console

Settlement Ledgers

David Chen

Customer

Commercial Settlement Ledger

2 Transactions

My Consignments

Book Shipment

Settlements & Invoices

Shipper Profile

Settlement & Billing

Review verified payment history, audit invoices, and settlement receipts.

Commercial Settlement Ledger

Audit-ready transactions and electronic payment receipts.

TRANSACTION REFERENCE	CONSIGNMENT	INSTRUMENT	AMOUNT	STATUS	TIMESTAMP	INVOICE
TXN-90218-VISA		Card	\$44.75	Pending Settlement	9/18/2025, 9:53:00 AM	View Receipt
TXN-90244-VLT		Wallet	\$25.00	Pending Settlement	9/9/2025, 3:53:00 PM	View Receipt

E.4 Customer – Settlements and Invoices

SwiftRoute

ENTERPRISE LOGISTICS

Public Tracking & Rates

Operations Console

Arthur Pendleton

ADMIN

SwiftRoute

Admin Hub

Shipper Enterprise Directory

SwiftRoute

CHIEF DISPATCHER

Consignment Dispatch

Commercial Shippers

Fleet Couriers

Revenue Analytics

System Audit Logs

Logistics Config

Fleet Administration & Dispatch Hub

Enterprise governance, courier allocations, freight analytics, and audit logging.

Total Consignments

4

+18.4% network throughput

Active Routing In Transit

1

Monitored on live route nodes

Delivered With Proof

0

Zero unresolved exceptions

Gross Freight Revenue

\$137.95

Net settled commercial funds

Registered Shippers & Corporate Accounts

Manage customer credentials and account activity status.

6 Shippers

SHIPPER ORGANIZATION	CONTACT EMAIL	PHONE NUMBER	DEFAULT DISPATCH ADDRESS	STATUS	ACCESS CONTROL
David Chen	customer@swiftroute.com	+1 (555) 345-6789	582 Market St, Floor 14, San Francisco, CA 94104	ACTIVE	Suspend
Sophia Martinez	sophia.m@gmail.com	+1 (555) 901-2345	1080 Folsom St, Apt 3B, San Francisco, CA 94103	ACTIVE	Suspend
Shiva Tiwari	st5951323@gmail.com	N/A	N/A	ACTIVE	Suspend
Shivam Tiwari	st4929617@gmail.com	N/A	N/A	ACTIVE	Suspend
SHIVAM TIWARI	shivamt1wari@klh.edu.in	N/A	N/A	ACTIVE	Suspend
Shivam Tiwari	shivam161761@gmail.com	N/A	N/A	ACTIVE	Suspend

E.5 Administrator – Commercial Shippers

SwiftRoute

ENTERPRISE LOGISTICS

Public Tracking & Rates

Operations Console

Arthur Pendleton

ADMIN

SwiftRoute

Admin Hub

Courier Fleet Personnel

SwiftRoute

CHIEF DISPATCHER

Consignment Dispatch

Commercial Shippers

Fleet Couriers

Revenue Analytics

System Audit Logs

Logistics Config

Fleet Administration & Dispatch Hub

Enterprise governance, courier allocations, freight analytics, and audit logging.

Total Consignments

4

+18.4% network throughput

Active Routing In Transit

1

Monitored on live route nodes

Delivered With Proof

0

Zero unresolved exceptions

Gross Freight Revenue

\$137.95

Net settled commercial funds

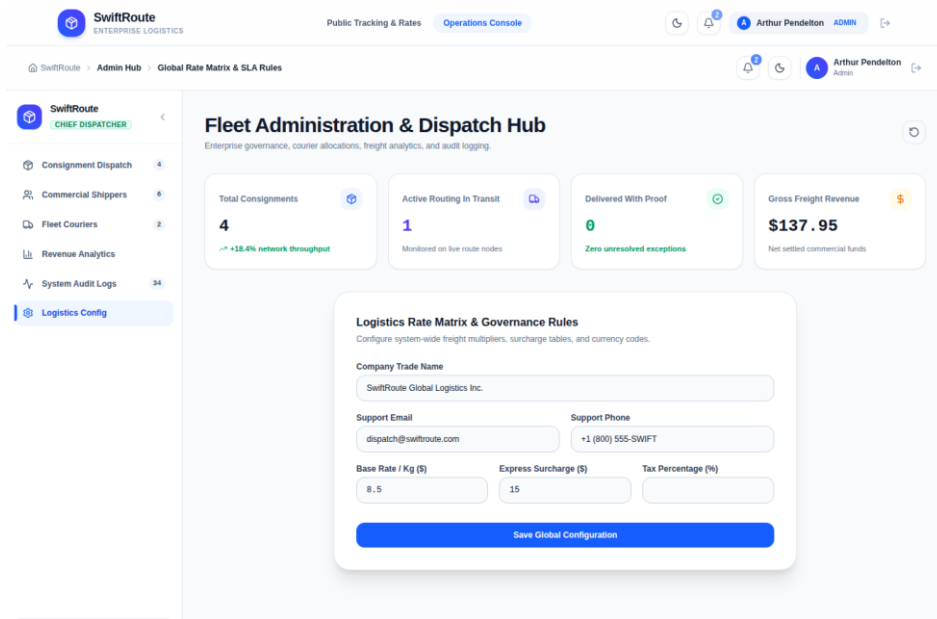
Courier Agent Personnel

Onboard drivers and monitor regional depot assignments.

Onboard New Courier

DRIVER NAME & DEPOT	CONTACT EMAIL	PHONE	ACTIVE ROUTE CONSIGNMENTS	STATUS	ACCESS CONTROL
Marcus Vance 32 Logistics Way, Bay Area Hub	agent_marcus@swiftroute.com	+1 (555) 234-5678	0 packages on van	ACTIVE	Suspend
Elena Rostova 44 Harbor Blvd, Logistics District	agent_elena@swiftroute.com	+1 (555) 876-5432	1 packages on van	ACTIVE	Suspend

E.6 Administrator – Fleet Couriers



E.7 Administrator – Logistics Config (rate matrix)

Appendix F: Complete Test Results

All 58 checks of the scripted API suite (apitest.py) with the expected and actual outcome.

#	Test case	Expected	Actual	Result
1	Health check	200	200	Pass
2	Login with wrong password	401	401	Pass
3	Admin login with seeded credentials	200 + JWT	200	Pass
4	Customer login	200 + JWT	200	Pass
5	Agent login	200 + JWT	200	Pass
6	Login with missing password	400	400	Pass
7	Protected route without token	401	401	Pass
8	Protected route with invalid token	401	401	Pass
9	Get current user with valid token	200	200	Pass
10	Customer calls admin-only route	403	403	Pass
11	Agent calls admin-only route	403	403	Pass
12	Admin dashboard statistics	200	200	Pass
13	Register with 3-char password	422	422	Pass
14	Register with invalid email	422	422	Pass
15	Register with duplicate email	409	409	Pass
16	Register new customer	201	201	Pass
17	Register supplying role=admin in body	403/ignored (expected secure behaviour)	201 role=admin	Fail (D1)
18	Book parcel with missing fields	422	422	Pass
19	Agent attempts to book parcel	403	403	Pass
20	Book standard parcel (3 kg)	201, cost 25.50, status pending	201, cost 25.5, pending	Pass
21	Tracking number format SR-YYYYLL-NNNNNN	matches regex	SR-2026TU-716966	Pass
22	Express parcel cost (2 kg)	17.00 + 15.00 = 32.00	32	Pass
23	Fragile parcel cost (1 kg)	8.50 + 7.50 = 16.00	16	Pass

#	Test case	Expected	Actual	Result
24	Minimum charge for very light parcel (0.2 kg)	5.00	5	Pass
25	Public tracking of new parcel (no login)	200 + 1 checkpoint	200, 1 checkpoint(s)	Pass
26	Tracking number lookup is case-insensitive	200	200	Pass
27	Track unknown tracking number	404	404	Pass
28	New customer views another customer's parcel	403	403	Pass
29	Owner views own parcel	200	200	Pass
30	Customer attempts to assign agent	403	403	Pass
31	Assign agent without agent_id	400	400	Pass
32	Assign non-existent agent	error (4xx/5xx), no state change	500	Pass (see D3)
33	Admin assigns courier agent	200, status assigned	200, assigned	Pass
34	Customer attempts status update	403	403	Pass
35	Status update without status value	400	400	Pass
36	Agent updates status to picked_up	200	200	Pass
37	Agent updates status to in_transit	200	200	Pass
38	Agent updates status to out_for_delivery	200	200	Pass
39	Timeline has one checkpoint per transition	5 (booked, assigned, 3 updates)	5	Pass
40	Status update with value outside the allowed set	400/422 (expected secure behaviour)	200	Fail (D2)
41	Checkout without parcel_id	400	400	Pass
42	Pay for parcel	201, payment completed	201	Pass
43	Pay for the same parcel twice	400, already paid	400	Pass
44	Invoice total = cost + 8.25% tax	25.50 + 2.10 = 27.60	27.6	Pass
45	Customer attempts to submit delivery proof	403	403	Pass
46	Agent submits proof of delivery	201	201	Pass
47	Parcel becomes delivered and proof stored	delivered + proof present	delivered, proof=yes	Pass
48	Search parcels by tracking number	1 result	1	Pass
49	Filter parcels by status	all delivered	2	Pass
50	Admin lists users	200	200	Pass
51	Admin reports	200	200	Pass
52	Admin activity (audit) log	200	200	Pass
53	Admin onboards courier agent	201	201	Pass
54	Admin suspends an agent	200	200	Pass
55	Suspended user login	403	403	Pass
56	Invalid user status value	400	400	Pass
57	Admin changes base rate	200	200	Pass
58	New rate applied to next booking (3 kg x 10)	30.00	30	Pass